| Data set | Size | Covariates | Unique Durations | Prop. Censored |
|---|---|---|---|---|
| SUPPORT | 8,873 | 14 | 1,714 | 0.32 |
| METABRIC | 1,904 | 9 | 1,686 | 0.42 |
| Rot. & GBSG | 2,232 | 7 | 1,230 | 0.43 |
| FLCHAIN | 6,524 | 8 | 2,715 | 0.70 |

Table 1: Summary of the four data sets used in the experiments in Section 6.1.

cessing. The data sets include the Study to Understand Prognoses Preferences Outcomes and Risks of Treatment (SUPPORT), the Molecular Taxonomy of Breast Cancer International Consortium (METABRIC), and the Rotterdam tumor bank and German Breast Cancer Study Group (Rot. & GBSG). Katzman et al. (2018) also provide the Worcester Heart Attack Study (WHAS) data set. However, their version of the data set is actually a case-control data set, meaning it contains multiple replications of some individuals, something the authors seem to have overlooked. We replace it with the Assay Of Serum Free Light Chain (FLCHAIN) made available in the survival packages of R (Therneau, 2015). For FLCHAIN, we remove individuals with missing values. Further, we remove the "chapter" covariate, which gives the cause of death. Table 1 provides a summary of the data sets. For a more detailed description, we refer to the original sources (Therneau, 2015; Katzman et al., 2018).

#### 6.1.1 Methods and Hyperparameter Tuning

In Sections 3.3 and 3.4 we presented two new survival methods based on case-control sampling and neural networks: a proportional Cox method and a non-proportional Cox method, which we will refer to as *Cox-MLP (CC)* and *Cox-Time* respectively. We will compare our methods to a classical linear Cox regression referred to as *Classical Cox (Linear)*, DeepHit (Lee et al., 2018), and Random Survival Forests (RSF) (Ishwaran et al., 2008). We will also compare to a proportional Cox method similar to DeepSurv (Katzman et al., 2018), but our version performs batched SGD by computing the negative partial log-likelihood in (5) on a subset of the data set. Furthermore, we choose not to restrict the network structure and optimization scheme to that of Katzman et al. (2018). Hence, this method is identical to our proportional Cox method in Section 3.3, except that it computes the negative partial log-likelihood of a batch, while we use case-control sampling in the loss function. We will refer to these two methods as Cox-MLP (DeepSurv) and Cox-MLP (CC) respectively. We do not compare with Luck et al. (2017) as their method is another proportional hazard method and is therefore restricted in all the same ways as our other proportional methods. As we will show in Section 6.1.2, the proportional hazards assumption is very restrictive, and methods based on this assumption are therefore not able to compete with other methods such as DeepHit, RSF, and Cox-Time.

For the neural networks, we standardize the numerical covariates and encode the categorical covariates by entity embeddings (Guo and Berkhahn, 2016) half the size of the number of categories. For the Classical Cox (Linear) regression, we one-hot encoded the categorical covariates (dummy variables), and in RSF we simply passed the covariates without any transformations. The networks are standard multi-layer perceptrons with the same

| Method | SUPPORT | METABRIC | Rot. & GBSG | FLCHAIN |
|---|---|---|---|---|
| Classical Cox (Linear) | 0.598 | 0.628 | 0.666 | 0.790 |
| Cox-MLP (DeepSurv) | 0.611 | 0.636 | 0.674 | 0.790 |
| Cox-MLP (CC) | 0.613 | 0.643 | 0.669 | **0.793** |
| Cox-Time | 0.629 | 0.662 | **0.677** | 0.790 |
| DeepHit | **0.642** | **0.675** | 0.670 | 0.792 |
| RSF (Mortality) | 0.628 | 0.649 | 0.667 | 0.784 |
| RSF ($C^{\text{td}}$) | 0.634 | 0.652 | 0.669 | 0.786 |

Table 2: Concordance ($C^{\text{td}}$) for the experiments in Section 6.1.

number of nodes in every layer, ReLU activations, and batch normalization between layers. We used dropout, normalized decoupled weight decay (Loshchilov and Hutter, 2019), and early stopping for regularization. SGD was performed by AdamWR (Loshchilov and Hutter, 2019) with an initial cycle length of one epoch, and we double the cycle length after each cycle. Learning rates were found using the methods proposed by Smith (2017).

As the four data sets are somewhat small, we scored our fitted models using 5-fold cross-validation, where the hyperparameter search was performed individually for each fold. For all the neural networks, we performed a random hyperparameter search over 300 parameter configurations and chose the model with the best score on a held-out validation set. We scored the proportional Cox methods by the partial log-likelihood, and for the Cox-Time method, we used the loss (12). Model hyperparameter tuning for DeepHit and RSF was performed with the time-dependent concordance index (Antolini et al., 2005). However, we also include an RSF tuned in the manner proposed by the authors, i.e., by computing the concordance of the mortality (Ishwaran et al., 2008, Section 4.1). The two versions of RSF are in the following denoted RSF ($C^{\text{td}}$) and RSF (Mortality), respectively. A list of the hyperparameter search spaces can be found in Appendix A.1.

#### 6.1.2 Results

We compare the methods using the metrics presented in Section 4, i.e., the time-dependent concordance, the integrated Brier score, and the integrated binomial log-likelihood. While the concordance solely evaluates a method's discriminative performance, the Brier score and binomial log-likelihood also evaluate the calibration of the survival estimates.

Table 2 shows the time-dependent concordance, or $C^{\text{td}}$, averaged over the five cross-validation folds. As expected, the methods that assume proportional hazards, in general, perform worse than the less restrictive methods, and Cox-MLP (DeepSurv) and Cox-MLP (CC) are very close to each other. We see that RSF ($C^{\text{td}}$) has better concordance than RSF (Mortality), which is expected as RSF ($C^{\text{td}}$) uses the same metric for hyperparameter tuning as for evaluation. Cox-Time seems to do slightly better than the RSF methods, which is impressive as we have not used concordance for hyperparameter tuning. DeepHit seems to have the best discriminative performance overall, but, as we will see next, this comes at the cost of poorly calibrated survival estimates.

| Method | SUPPORT | METABRIC | Rot. & GBSG | FLCHAIN |
|---|---|---|---|---|
| Classical Cox (Linear) | 0.217 | 0.183 | 0.180 | 0.096 |
| Cox-MLP (DeepSurv) | 0.214 | 0.176 | 0.170 | **0.092** |
| Cox-MLP (CC) | 0.213 | 0.174 | 0.171 | 0.093 |
| Cox-Time | **0.212** | **0.172** | **0.169** | 0.102 |
| DeepHit | 0.223 | 0.184 | 0.178 | 0.120 |
| RSF (Mortality) | 0.215 | 0.175 | 0.171 | 0.093 |
| RSF ($C^{\text{td}}$) | **0.212** | 0.176 | 0.171 | 0.093 |

Table 3: Integrated Brier score weighted by estimates of the censoring distribution for the experiments in Section 6.1.

| Method | SUPPORT | METABRIC | Rot. & GBSG | FLCHAIN |
|---|---|---|---|---|
| Classical Cox (Linear) | -0.623 | -0.538 | -0.529 | -0.322 |
| Cox-MLP (DeepSurv) | -0.619 | -0.532 | -0.514 | **-0.309** |
| Cox-MLP (CC) | -0.615 | -0.515 | -0.509 | -0.314 |
| Cox-Time | -0.613 | **-0.511** | **-0.502** | -0.432 |
| DeepHit | -0.637 | -0.539 | -0.524 | -0.487 |
| RSF (Mortality) | -0.619 | -0.515 | -0.507 | -0.311 |
| RSF ($C^{\text{td}}$) | **-0.610** | -0.517 | -0.507 | -0.311 |

Table 4: Integrated binomial log-likelihood weighted by estimates of the censoring distribution for the experiments in Section 6.1.

Tables 3 and 4 show the integrated Brier score and integrated binomial log-likelihood, both weighted with Kaplan-Meier estimates of the censoring distribution. Here, for both metrics, closer to zero is better. We find that both metrics yield very similar results, as the orderings of the methods are almost identical. First, we see that Cox-Time seems to generally perform the best, but it struggles with the FLCHAIN data set. However, we note that, for FLCHAIN, Cox-MLP (DeepSurv) has the best integrated Brier score and binomial log-likelihood while Cox-MLP (CC) has the best concordance. This indicates that the proportionality assumption is quite reasonable for this data set.

Again, we find that there is very little difference between Cox-MLP (DeepSurv) and Cox-MLP (CC), which is as expected. The RSF methods generally perform well. Note that the two RSF methods perform equally well here, even though RSF ($C^{\text{td}}$) had the best concordance. Classical Cox (Linear) does rather poorly, which was expected as it has very restrictive model assumptions.

Even though DeepHit, in general, had the best concordance, it has the worst integrated Brier score and binomial log-likelihood in three out of the four data sets. The loss function in DeepHit, given by formula (D.2) in Appendix D, is a convex combination of the negative log-likelihood and a ranking loss, determined by a parameter $\alpha$. For $\alpha = 1$ we obtain only the negative log-likelihood and for $\alpha = 0$ we obtain only the ranking loss. As we

| Data set | Size | Churned | Censored | Prop. Censor | Unique users |
|---|---|---|---|---|---|
| Train | 1,786,333 | 1,279,358 | 506,975 | 0.28 | 1,582,202 |
| Test | 661,748 | 473489 | 188,259 | 0.28 | 586,001 |
| Validation | 198,665 | 142,104 | 56,561 | 0.28 | 175,801 |

Table 5: Summary of the KKBox churn data set.

do hyperparameter tuning based on the concordance, we see that $\alpha$ tends towards smaller values, which results in excellent discriminative performance at the cost of poorly calibrated survival estimates.

### 6.2 KKBox Churn Case Study

Thus far we have compared competing survival methodologies on fairly small data sets. We now perform a case study on a much larger data set, as this is more interesting in the context of neural networks.

The WSDM KKBox's churn prediction challenge was proposed in preparation for the 11th ACM International Conference on Web Search and Data Mining.[2] The competition was hosted by Kaggle in 2017, with the goal of predicting customer churn on a data set donated by KKBox, the leading music streaming service in Asia. The available data provide us the opportunity to create a survival data set with event times and censoring indicators. We stick with the churn definition given by KKBox, were a customer is considered churned if he or she fails to resubscribe within 30 days after the previous subscription expired. Note, however, that our use of the data is not comparable to the Kaggle competition, as we work with survival times from the start of a subscription period, while they consider durations from a fixed calendar date.

KKBox provides multiple data sources, but as we are primarily interested in evaluating our methods, we spend less time on feature engineering and only use a subset of covariates with general customer information (e.g., city, age, price of subscription). Furthermore, a customer that has previously churned and later resubscribed, is treated as a new customer with some covariate information describing the previous subscription history. This gives us a total of 15 covariates. We split the data into a training, a testing, and a validation set, and some information about these subsets are listed in Table 5. A more in-depth description of the KKBox data set can be found in Appendix B.1.

#### 6.2.1 Methods and Hyperparameter Tuning

We use the same methods as in Section 6.1. However, as this data set is very large, we replace the classical Cox regression with our Cox-SGD (Linear) method from Section 3.2.

We standardize and encode the covariates in the same manner as for the smaller data sets. The networks and training are also the same as earlier, but we multiply the learning rate by 0.8 at the start of every cycle, as we found this to give more stable training. The KKBox data set is quite large, so we are not able to explore as large a hyperparameter space as in the previous experiments. Hence, we do not include weight decay, and perform

2. `https://www.kaggle.com/c/kkbox-churn-prediction-challenge`